HYPERLOCALIZE

CONTEXT-AWARE IDENTITY-PRESERVING VIDEO LOCALIZATION

A PROJECT REPORT

Submitted by

[STUDENT NAME] ([REGISTER NUMBER])

in partial fulfilment of the requirements for the award of the degree of

[BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY]

in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

[GUIDE NAME], [DESIGNATION]

[DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING]

[COLLEGE NAME]

[UNIVERSITY NAME]

[MONTH YEAR]

BONAFIDE CERTIFICATE

Certified that this project report **"HyperLocalize: Context-Aware Identity-Preserving Video Localization"** is the bonafide work of **[STUDENT NAME] ([REGISTER NUMBER])**, who carried out the project work under my supervision.

SIGNATURE
[HEAD OF THE DEPARTMENT]
[Department, College]

SIGNATURE
[SUPERVISOR / GUIDE]
[Designation, Department]

Submitted for the project viva-voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the project report entitled **"HyperLocalize: Context-Aware Identity-Preserving Video Localization"**, submitted in partial fulfilment of the requirements for the award of the degree of [B.E. / B.Tech.] in Computer Science and Engineering, is a record of original work carried out by me under the guidance of [GUIDE NAME]. Third-party software, models, datasets and published material used in this work are acknowledged in the References and in the repository file `docs/THIRD_PARTY_TERMS.md`.

Software development in this project was assisted by an AI coding assistant (Claude, by Anthropic). Design decisions were approved by me and recorded as Architecture Decision Records; the assistant's role and the rules that limit it (for example, that it may never perform human review or approval) are described in Section 6.7.

Place: [PLACE] **[STUDENT NAME]**
Date: [DATE] [REGISTER NUMBER]

ACKNOWLEDGEMENT

I express my sincere gratitude to [PRINCIPAL NAME], Principal, [COLLEGE NAME], for providing the facilities to carry out this project.

I thank [HOD NAME], Head of the Department of Computer Science and Engineering, for the encouragement and support given throughout the project.

I am deeply grateful to my project guide, [GUIDE NAME], [DESIGNATION], for the guidance, review and constructive criticism that shaped the scope and honesty of this work.

I also thank the faculty members and staff of the department, and my family and friends, for their support. I acknowledge the authors and maintainers of the open-source software, models and public-domain or Creative Commons videos used in this project.

[STUDENT NAME]

ABSTRACT

Important video information, for example in health, food, water and energy, often fails to reach people because it is published in a language or cultural framing they do not share. Automatic dubbing pipelines (speech recognition, then machine translation, then speech synthesis) can produce a localized video quickly, but they treat the AI output as final: a dropped "not", a changed dosage number or an unconsented copy of a speaker's voice can reach the audience unnoticed.

HyperLocalize is a context-aware, identity-preserving video localization system built as a **controlled workflow** rather than an autonomous pipeline. A Localization Manager creates a project with a target-audience profile and glossary; the source video is validated and analysed into scenes and timed segments; each segment is localized with its context (neighbouring lines, speaker, audience profile, glossary, timing budget and reviewer instruction); a preview is rendered; and a human reviewer edits, overrides with a written reason, or regenerates a single scene. Rule-based checks on critical content (numbers, units, glossary terms, structure and speech overrun) block approval until resolved. A speaker's identity is used only while that person's own versioned consent is valid; revocation stops running jobs, quarantines and deletes derived audio, and falls back to a stock voice. Approval freezes an immutable snapshot, export refuses any placeholder (MOCK) output, and every governance action is written to an append-only, SHA-256 hash-chained audit log whose head is embedded in the export's evidence manifest.

The system is implemented as a Django modular monolith (nine apps) on PostgreSQL 17 with a hand-written job queue, server-rendered pages with htmx, and AI stages behind swappable provider interfaces. Candidate real providers are faster-whisper (English transcription, CPU), Gemini (context-aware translation for scripted or licence-recorded published content) and MMS-TTS Tamil (stock voice, CPU); voice cloning is still a clearly labelled mock. The current demo scenario is English health information localized into Tamil.

Measured results (2026-10-22): the automated suite of 210 tests passes (2 optional real-model tests skipped); an end-to-end HTTP check passed 15 of 15 steps (2026-10-06); and 11 black-box tests (equivalence partitioning and boundary values) pass. In one pre-evaluation trial on a single 58-second public video, the voice-intelligibility proxy

(Whisper back-transcription character error rate) was 0.16 and only 2 of 14 lines fitted their time slot, which led to a length-budgeted prompt and a blocking overrun check. Translation quality, speaker similarity and the benefit of context have **not yet been measured**; a formal evaluation (spike) with consented speakers and a Tamil-literate rater is planned.

Keywords: *video localization, dubbing, context-aware machine translation, consent management, human-in-the-loop review, audit trail, hash chain, Tamil, Django.*

TABLE OF CONTENTS

BONAFIDE CERTIFICATE....2
DECLARATION....3
ACKNOWLEDGEMENT....4
ABSTRACT....5
TABLE OF CONTENTS....7
LIST OF FIGURES....9
LIST OF TABLES....10
INTRODUCTION....11
1.1 Background....11
1.2 Problem Statement....11
1.3 Objectives....12
1.4 Scope of the Project....12
1.5 Positioning....13
1.6 Methodology....13
1.7 Organisation of the Report....14
LITERATURE SURVEY AND EXISTING SYSTEMS....15
2.1 Automatic Speech Recognition....15
2.2 Machine Translation for Indian Languages....15
2.3 Text-to-Speech and Voice Cloning....15
2.4 Commercial Dubbing Tools....16
2.5 Translation Quality Assessment....16
2.6 Provenance and Audit....16
2.7 Research Gap and Differentiation....16
SYSTEM ANALYSIS AND REQUIREMENTS....18
3.1 Existing System and its Limitations....18
3.2 Proposed System....18
3.3 Feasibility Study....18
3.4 Actors....19
3.5 Functional Requirements....19
3.6 Non-Functional Requirements....20
3.7 Hardware and Software Requirements....20
SYSTEM DESIGN....22
4.1 System Architecture....22
4.2 Module Design....22
4.3 Use Case Diagram....24
4.4 Sequence Diagrams....24
4.5 State Diagrams....25
4.6 Data Model....27
4.7 Key Design Decisions....29
4.8 Consent Design....29
4.9 Audit Design....30
IMPLEMENTATION....31
5.1 Technology Stack....31
5.2 Asynchronous Job Pipeline....31
5.3 Video Analysis and Scene Planning....31
5.4 Context-Aware Localization....32
5.5 Critical-Content Checks....32

5.6 Rendering and Timing Fit....33
5.7 Consent Gate, Revocation and Mid-Job Stop....33
5.8 Review, Approval and Export Gates....34
5.9 Provider Abstraction and Mocks....34
5.10 User Interface....35
TESTING....39
6.1 Testing Strategy....39
6.2 Test Areas....39
6.3 Sample Test Cases....40
6.4 Black-Box Testing....41
6.5 Test Results....41
6.6 Not Yet Tested....42
6.7 Governance of Test Runs....42
RESULTS AND DISCUSSION....43
7.1 Evaluation Plan....43
7.2 First Real-Clip Trial (n = 1)....43
7.3 Design Changes Driven by the Results....44
7.4 Discussion....45
CONCLUSION AND FUTURE WORK....46
8.1 Conclusion....46
8.2 Limitations....46
8.3 Future Work....47
8.4 Viva Summary....47
REFERENCES....48
APPENDIX A: ARCHITECTURE DECISION RECORDS....49
APPENDIX B: CLASS DIAGRAM....50
APPENDIX C: USE-CASE STATUS....52
APPENDIX D: RUNNING THE SYSTEM....53
APPENDIX E: ABBREVIATIONS....54

LIST OF FIGURES

Figure 4.1: Component diagram (actual Django apps and processes)
Figure 4.2: Use case diagram (UC1–UC9 and Record Consent)
Figure 4.3: Sequence diagram: main localization workflow
Figure 4.4: Sequence diagram: revocation while a cloned-voice job runs
Figure 4.5: State diagram: Scene
Figure 4.6: State diagram: Localization (project output)
Figure 4.7: State diagram: Job
Figure 4.8: State diagram: Consent version (append-only rows)
Figure 4.9: Core entity-relationship diagram
Figure 5.1: Localization Manager: project list
Figure 5.2: Localization Manager: project detail with approval gate
Figure 5.3: Reviewer: scene review with preview, checks and override fields
Figure 5.4: Speaker / Identity Owner: consent page
Figure 5.5: Auditor: audit trail and chain verification
Figure 5.6: Administrator: policies, provider chains and quota warnings
Figure 7.1: Clip 01 timing: Tamil speech length ÷ slot length per line (1.0 = fits)
Figure B.1: Full class diagram (generated from the code)

LIST OF TABLES

Table 1.1: MVP scope
Table 2.1: Novelty classification of HyperLocalize features
Table 3.1: Feasibility study
Table 3.2: Actors and responsibilities
Table 3.3: Functional requirements by use case
Table 3.4: Hardware and software requirements
Table 4.1: Modules (Django apps) and their responsibilities
Table 4.2: Core entities
Table 4.3: Summary of key ADRs
Table 4.4: Consent checkpoints (ADR 0006)
Table 5.1: Critical-content checks (ADR 0008, ADR 0022)
Table 5.2: Provider chains and labelled mocks
Table 6.1: Test areas
Table 6.2: Sample test cases (all part of the passing suite)
Table 6.3: Measured test results
Table 7.1: Evaluation metrics and MEASURED results (targets are fixed in SPIKE_PLAN §5)
Table 7.2: Problems found and design changes (ADR 0022)

CHAPTER 1
INTRODUCTION

1.1 Background

Video has become one of the main ways in which public information is shared: health departments publish hand-washing and rehydration videos, agencies publish water-safety and energy-saving guidance, and organisations publish training material. Most of this material is produced in a small number of languages. For a viewer who does not understand the language, or for whom the framing, units or examples are unfamiliar, the information is effectively inaccessible.

Localization is more than translation. It adapts content for a target audience: the language, the level of formality and reading level, cultural references, units of measure and the timing of speech against the picture. When the content carries critical facts (a quantity, a time, a warning or a negation) an error in localization can change the meaning of the message.

Recent speech and language models make automated dubbing technically easy: a speech recogniser produces a transcript, a translation model or large language model (LLM) produces the target text, and a text-to-speech (TTS) model, sometimes a voice-cloning model, produces the new audio. This raises three practical problems that this project addresses: **(1) trust** in machine output for critical information, **(2) identity and consent** when a real speaker's voice is reproduced, and **(3) accountability** for who reviewed, approved and exported what.

1.2 Problem Statement

Existing automatic dubbing approaches treat localization as a one-shot pipeline whose output is assumed to be correct. They give the reviewer little support for finding the errors that matter most (numbers, units, terminology and meaning reversals), do not usually enforce a speaker's consent at every processing step, and produce an output file with no verifiable record of how it was made.

The problem addressed by this project is to design and build a controlled video localization workflow in which AI output is only a draft, critical content is checked automatically, a human reviews and approves every scene before export, a speaker's identity is used only while their consent is valid, and every governance

action is recorded in a tamper-evident audit trail that travels with the exported package.

1.3 Objectives

1. Provide a project-based workflow: Create Localization Project → Upload and Analyze Source Video → Context-Aware Localization → Generate Localized Preview → Human Review → Regenerate Scene/Output if Required → Approve → Export Localization Package.
2. Localize each segment using its **source and audience context** (neighbouring segments, speaker, audience profile, glossary, timing budget and reviewer instruction), and record which context inputs were used.
3. Detect errors in **critical content** automatically (numbers, units, glossary terms, structure, speech overrun) and block approval until a reviewer fixes the text or overrides it with a written reason.
4. Support **scene-level regeneration** so that a reviewer can redo one problematic scene without touching the rest.
5. Enforce **consent in the backend** before any use of a speaker's identity, and make **revocation** stop new generation, cancel or kill running jobs, and invalidate affected approvals and exports.
6. Gate **approval and export**: freeze an immutable snapshot, apply separation of duties, and refuse to export any placeholder (MOCK) output.
7. Record every consent grant, revocation, approval, regeneration and export in an **append-only, hash-chained audit log**, and include the chain head in an evidence manifest with every export.
8. Keep AI stages behind **provider interfaces** so that models can be swapped, mocked and administered, within a **zero-rupee budget**.

1.4 Scope of the Project

HyperLocalize is a general localization workflow, not a single-domain or single-language tool. For the first demonstrable build (the MVP), the scope was deliberately narrowed as shown in Table 1.1.

Table 1.1: MVP scope

Item	MVP decision	Source
Language pair	One pair, one target locale per project: English → Tamil	Product definition §7
Upload limits	Up to 5 minutes and 500 MB per video	ADR 0004

Item	MVP decision	Source
Face processing / lip-sync	Not included (future scope only, needs its own consent design)	Product definition §7, §10
Identity in the MVP	A consented reference voice plus versioned consent records, with stock-voice or subtitles-only fallback (provisional)	ADR 0015 (pending)
Deployment	Demo on one laptop	ADR 0012
Budget	₹0: free tiers and open-source models only	ADR 0005
Demo scenario	English health information (ORS, hand-washing) → Tamil. This is a test case, not the product's domain	Product definition §9

1.5 Positioning

HyperLocalize is positioned as an **information-access layer**. It can help important video information in domains such as health, food, water and energy cross language and cultural barriers. It does not solve the underlying physical problems in those domains, and it is not presented as a medical, nutritional or engineering tool. It is also explicitly **not** a voice-cloning product, not a competitor on raw dubbing quality, and not an autonomous pipeline.

1.6 Methodology

The project followed a design-first, evidence-first process:

- **Phase 0 (discovery and design):** the requirements, actors and use cases were analysed, and every major decision (architecture, database, job queue, consent model, audit design, data model and so on) was compared against 2–4 alternatives and recorded as an **Architecture Decision Record (ADR)**. 22 ADRs exist (Appendix A).
- **Phase 1 (MVP build):** the system was built in **vertical slices**: one thin path end to end first, then widened. Real AI providers are added only as candidates until a formal evaluation (the "spike") measures them.
- **Honesty labels:** every claim in the project documents is labelled [CONFIRMED], [PROPOSED], [INFERRED] or [UNKNOWN], and every metric is reported as a TARGET or a MEASURED RESULT. Mock components are named as mocks and can never be exported. This report follows the same rules.

- **Independent review:** ADRs, consent and audit code were reviewed by a separate, read-only reviewer with a fresh context; findings and fixes are recorded in `docs/ADR_REVIEW.md` and `docs/CODE_REVIEW.md`.

1.7 Organisation of the Report

Chapter 2 surveys related work and existing systems. Chapter 3 analyses the requirements. Chapter 4 presents the system design with UML diagrams. Chapter 5 describes the implementation and its algorithms. Chapter 6 covers testing. Chapter 7 presents results and discussion. Chapter 8 concludes and lists future work. The appendices list the ADRs, the full class diagram, the user guide and viva notes.

CHAPTER 2
LITERATURE SURVEY AND EXISTING SYSTEMS

This chapter reviews the building blocks of automatic video localization and the existing systems that use them. **Note on scope:** a systematic literature and product review has **not** yet been carried out; the survey below is based on well-known published work and on general knowledge of commercial tools, and product capabilities should be verified before they are quoted.

2.1 Automatic Speech Recognition

Whisper (Radford et al., 2022) is an encoder–decoder Transformer trained on about 680,000 hours of weakly supervised multilingual audio. It is robust to accents and noise and produces timed segments. faster-whisper re-implements Whisper inference with CTranslate2, which makes CPU inference practical on a laptop. HyperLocalize uses faster-whisper `small` on CPU as its candidate English transcriber, and also reuses it to transcribe its own Tamil speech back as an intelligibility proxy.

2.2 Machine Translation for Indian Languages

IndicTrans2 (Gala et al., 2023) is an open multilingual translation model covering the 22 scheduled Indian languages, including Tamil. It translates sentence by sentence. Large language models such as Gemini can translate with additional context supplied in the prompt (neighbouring sentences, glossary, audience and length constraints) and can explain their choices, but they may hallucinate or omit content, and free tiers have strict request quotas. HyperLocalize treats both as candidates behind one interface; Gemini is the current candidate for context-aware translation, and IndicTrans2 is planned as an offline baseline.

2.3 Text-to-Speech and Voice Cloning

The Massively Multilingual Speech (MMS) project (Pratap et al., 2023) released TTS models for more than 1,100 languages, built on the VITS architecture (Kim et al., 2021). The Tamil model `facebook/mms-tts-tam` runs on CPU and is used here as the stock voice (licence CC-BY-NC-4.0, non-commercial). Zero-shot voice-cloning models can imitate a speaker from a short reference clip; Indian-language candidates such as

IndicF5 exist, but cloning raises consent and misuse risks. In this project cloning is behind a consent gate and is currently a mock.

2.4 Commercial Dubbing Tools

Commercial AI dubbing products (for example ElevenLabs dubbing, HeyGen and Rask) and platform features such as multi-language audio tracks on video platforms already offer transcription → translation → voice cloning, transcript editing, segment re-generation and in some cases lip-sync. Some require attestation or verification of voice-cloning consent. These are capable tools; HyperLocalize does not try to compete with them on raw dubbing quality.

2.5 Translation Quality Assessment

The Multidimensional Quality Metrics (MQM) framework (Lommel et al., 2014) scores translations by error category and severity, and is the basis of the human evaluation planned for this project. chrF (Popović, 2015) is a character n-gram F-score that works better than word-level metrics for morphologically rich languages such as Tamil. Rule-based quality-assurance checks for numbers, tags and terminology are standard features of computer-assisted translation tools.

2.6 Provenance and Audit

The C2PA specification defines signed provenance manifests for media. Hash chains, in which each record stores the hash of the previous one, are a long-established way to make a log tamper-evident. HyperLocalize uses a hash-chained audit log and an evidence manifest; it does not implement C2PA signing.

2.7 Research Gap and Differentiation

No single component of HyperLocalize is new. Table 2.1 classifies each feature honestly, following the project's novelty rules.

Table 2.1: Novelty classification of HyperLocalize features

Feature	Classification	Note
ASR → MT → TTS pipeline; voice cloning	Existing / Common	Commodity, swappable
Transcript editing, per-segment regeneration, human review	Existing / Common	Expected in professional localization

Feature	Classification	Note
Provider abstraction, mocks, Postgres job queue, modular monolith	Implementation Choice	Good engineering, not novelty
LLM translation with audience profile, glossary, timing budget and rationale	Enhancement	Context-aware LLM translation is widely researched
Rule-based hard checks as an approval-blocking gate	Enhancement	Similar QA checks exist in CAT tools
AI meaning-reversal flags that require acknowledgement	Enhancement	An aid to review only
Versioned, backend-enforced consent with a revocation cascade and fallback	Enhancement	A required safeguard, done carefully
Evidence-carrying export with audit chain head	Enhancement	Provenance manifests (C2PA) exist
The combination: critical-content checks + unbypassable review gate + consent enforcement + evidence-carrying export	Proposed Novelty	Must be verified by a product and literature check before being claimed

The gap targeted by this project is therefore not a new model, but a **controlled, accountable workflow** around commodity models, aimed at information where errors and misuse of identity matter.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System and its Limitations

- Output is treated as final; review, if any, is optional and unstructured.
- No automatic detection of changed numbers, units or key terms, so a reviewer must find them by reading.
- Consent for voice reproduction is, at best, an attestation at upload time; revocation does not reliably stop work already in progress.
- No verifiable record of who reviewed and approved which version, or which models produced it.
- Tied to one vendor's models; cannot be run with free or local models.

3.2 Proposed System

The proposed system wraps the commodity pipeline in a governed workflow: context-aware drafting, rule checks that block approval, scene-level review and regeneration, a backend consent gate checked at every stage boundary, an approval gate with an immutable snapshot, and an export package that carries its own evidence. The AI stages are replaceable providers, and placeholder output is always labelled and never exportable.

3.3 Feasibility Study

Table 3.1: Feasibility study

Aspect	Assessment
Technical	Feasible on one laptop (Intel i5-13450HX, 10 cores / 16 threads, 14 GiB RAM, Ubuntu 26.04). Transcription and TTS run on CPU. The web/worker split keeps long jobs off the request path.
Economic	₹0 budget: open-source software and models, free-tier Gemini, local PostgreSQL. Constraint: the Gemini free tier allows about 20 requests per day per model (measured), so calls are batched.
Operational	Server-rendered web pages per role; no client installation. A demo can be reset and rebuilt with management commands.
Legal and ethical	Voices are personal data, so consent is enforced in the backend and revocable. Test videos are public-domain or CC BY 3.0 with recorded attribution. MMS-TTS is non-commercial. No compliance claim (for example GDPR) is made.

3.4 Actors

Table 3.2: Actors and responsibilities

Actor	Responsibilities
Localization Manager	Creates projects, sets audience and glossary, adds members, approves and exports
Reviewer / Linguist	Reviews scenes, edits text, overrides checks with reasons, acknowledges AI flags, requests regeneration, accepts scenes
Media Operator	Uploads source video and transcripts, links speakers to identity owners, monitors jobs
Speaker / Identity Owner	Grants, narrows, revokes or withdraws consent; uploads a reference clip
Administrator	Manages users, provider chains, policies and free-quota limits (cannot grant consent)
Auditor / Faculty Evaluator	Inspects the audit trail, verifies the hash chain, reviews overrides and evidence

3.5 Functional Requirements

Table 3.3: Functional requirements by use case

Use case	Main functional requirements
UC1 Create and Generate a Localization Project	Create a project with languages, audience profile and data classification; manage glossary and members; start the pipeline
UC2 Upload and Analyze Source Video	Validate the file before storing it (ffprobe; 5 min / 500 MB); extract audio; transcribe (human SRT/VTT, else ASR); detect shot cuts; plan scenes and segments
UC3 Generate Context-Aware Localization Preview	Localize each segment with its context; record context inputs, adapter, model and prompt version; run rule checks and AI flags
UC4 Generate Localized Preview	Resolve the rendering mode per speaker from consent; synthesise speech; fit timing; mix; produce a scene preview with subtitles
UC5 Review and Regenerate a Scene	Edit (new human version), override with reason, acknowledge flags, accept a scene, regenerate one scene's script or preview
UC6 Approve and Export Localization Package	Approve through the gate; freeze a snapshot; export MP4, SRT, VTT, script, quality report and evidence manifest; refuse MOCK output
UC7 Revoke Consent and Stop Identity	Grant, narrow, revoke or withdraw consent; cascade revocation to jobs, artefacts, approvals and exports; purge; fall back
UC8 Administer Users, Providers and Policies	Manage users and memberships, provider chain order, policies and free-quota warnings
UC9 Inspect Quality, Audit and Evidence	Filter the audit trail, verify the chain, list overrides
Record Consent («include»)	Record standing and project consent with a signed-form reference

3.6 Non-Functional Requirements

- **Responsiveness:** a web request never blocks on long work; video processing runs as asynchronous jobs with explicit states (queued, running, succeeded, failed, cancelled).
- **Safety of identity:** consent is re-checked at each stage boundary; a revocation must stop a running job within seconds (tested bound: under 8 seconds).
- **Integrity:** audit records cannot be updated or deleted through the database; any tampering must be detectable by recomputing the chain.
- **Honesty:** every AI or placeholder artefact is labelled with its adapter, model and `is_mock` flag; MOCK output can never be exported.
- **Privacy practice:** media stored outside the web root; secrets in environment variables; no raw content in logs; retention sweeps (previews 30 days, exports 90 days).
- **Maintainability and explainability:** a modular monolith whose apps call each other only through service functions; code a student can explain in a viva.
- **Reliability:** provider errors, timeouts, quota exhaustion and invalid uploads are handled explicitly; a provider failure never silently produces placeholder text.

3.7 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Category	Requirement (as used in development)
Processor	Intel Core i5-13450HX (10 cores, 16 threads)
Memory	14 GiB RAM (8 GB planned Docker memory limit)
Storage	About 1 GB for models (MMS-TTS ≈ 280 MB, Whisper small) plus media
Operating system	Ubuntu 26.04.1 LTS (Linux)
Language and framework	Python 3, Django 6.1.1 (no Django REST Framework)
Database	PostgreSQL 17 (psycopg 3.3.6)
Frontend	Server-rendered Django templates with htmx
Media tools	ffmpeg / ffprobe (static-ffmpeg package in development)
AI (optional, candidates)	faster-whisper 1.2.1; PyTorch 2.14.1 (CPU) with transformers for MMS-TTS Tamil; Gemini API (free tier)
Deployment	Docker Compose files written (not yet run, Docker not installed); development script with embedded PostgreSQL
Browser	Any modern web browser

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

HyperLocalize is a **modular monolith** (ADR 0001): one Django codebase and one PostgreSQL database, with a **web process** that serves pages and never blocks on long work, and **worker processes** that claim jobs from a hand-written PostgreSQL job queue (`run_worker --lane default|synthesis|all`). Heavy model inference runs in a separate, killable subprocess so that a consent revocation can stop it. Media lives outside the web root behind a storage interface. AI stages sit behind provider interfaces with an ordered fallback chain per stage. A monolith was chosen over microservices because the project has one developer, a laptop demo and a zero budget, and because database transactions and row locks give the strong guarantees that consent enforcement needs.

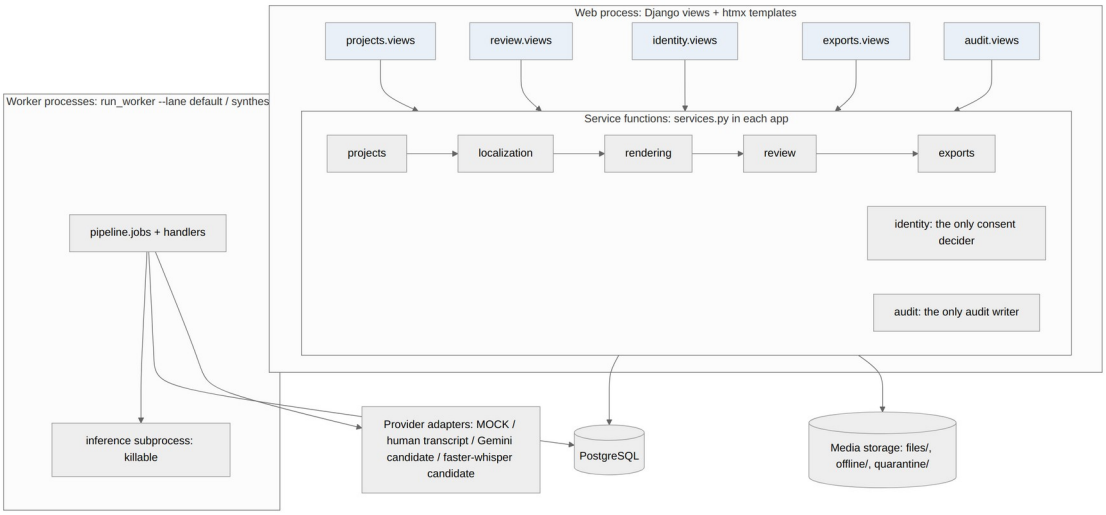


Figure 4.1: Component diagram (actual Django apps and processes)

4.2 Module Design

Table 4.1: Modules (Django apps) and their responsibilities

App	Owns	Key code
accounts	User, ProjectMembership roles, Policy values, rate-limited login	permissions.py: role checks used by every service
projects	Project, AudienceProfile, GlossaryTerm, Video, SpeakerTrack, media storage	services.upload_video (ffprobe validation); LocalMediaStorage
localization	Scene, Segment, Localization, SegmentTranslation (append-only versions), CheckResult	services.localize_scene; checks.py

App	Owns	Key code
rendering	SynthesisedSegment, RenderedScene	services.render_scene (mode resolution, consent checkpoints CP4–CP7)
review	Review, RegenerationRequest, Approval	services.approve (the gate), request_changes
identity	IdentityOwner, Consent, VoiceProfile, OfflineCopy	services.evaluate: the only consent decider
exports	Export	services.request_export, build_export
audit	AuditRecord, AuditChainHead	services.record: the only audit writer; verify_chain
pipeline	Job, ProviderConfig, ProviderUsage	jobs.py, handlers.py, inference.py, providers/

Apps call each other only through `services.py` functions, and Django signals are not used, so every cross-module call is explicit and easy to trace.

4.3 Use Case Diagram

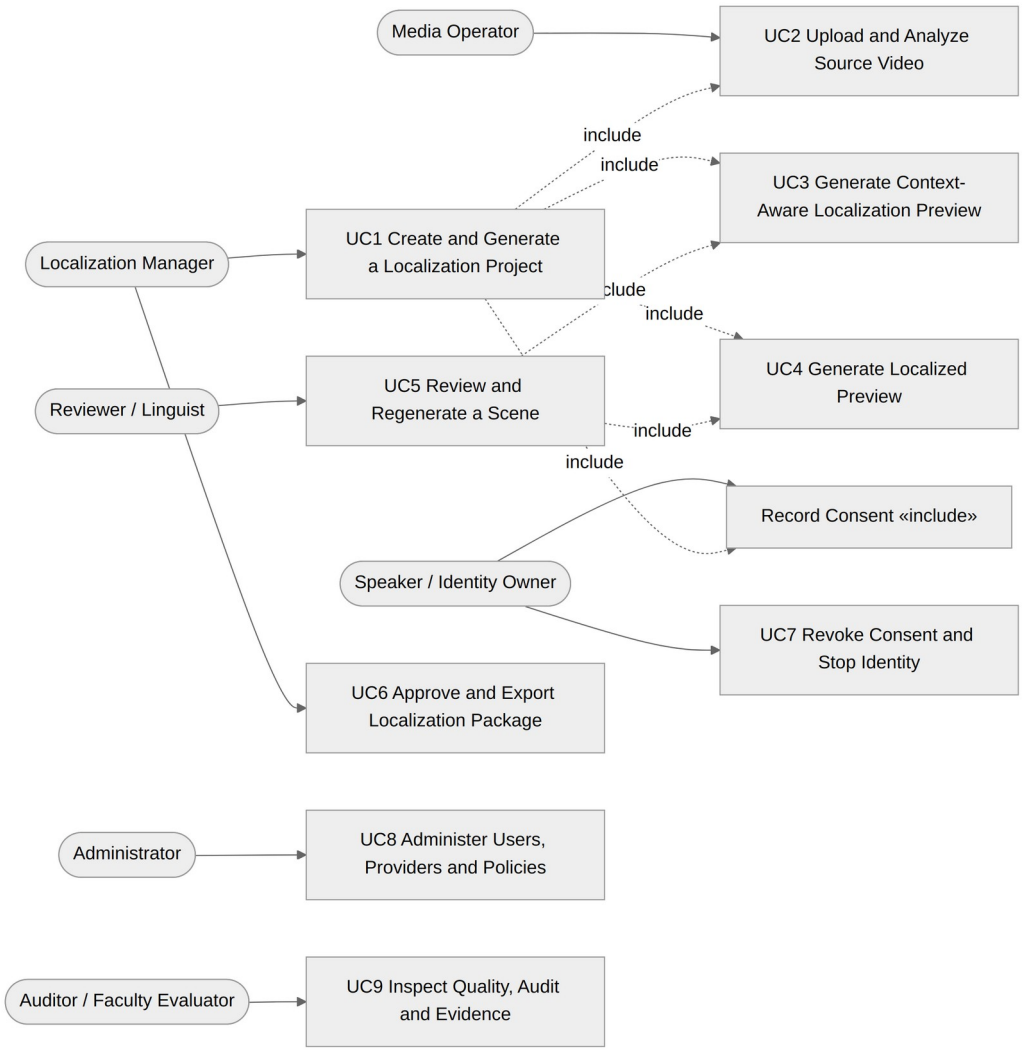


Figure 4.2: Use case diagram (UC1–UC9 and Record Consent)

4.4 Sequence Diagrams

Figure 4.3 shows the main workflow as implemented, from upload to export. Figure 4.4 shows what happens when a speaker revokes consent while a cloned-voice render job is running.

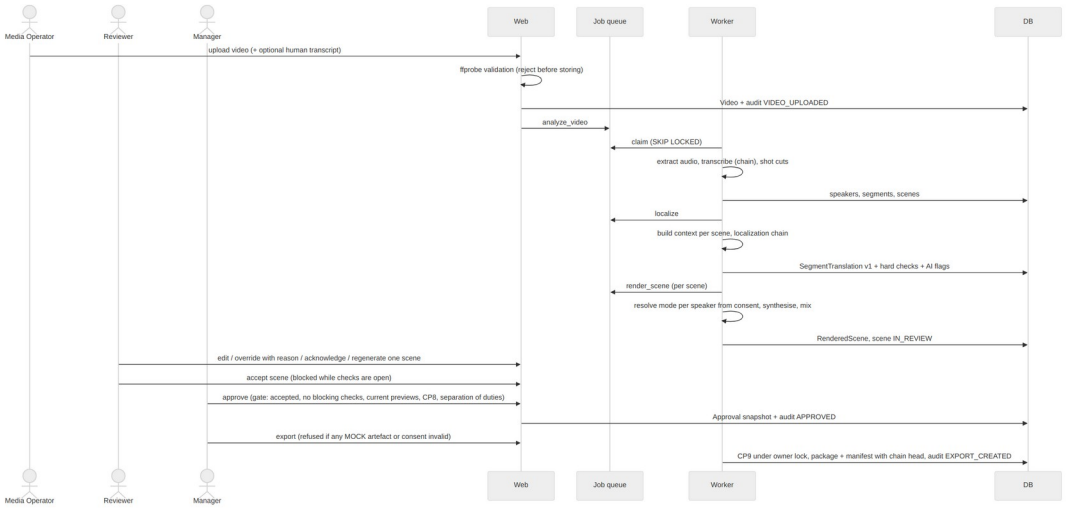


Figure 4.3: Sequence diagram: main localization workflow

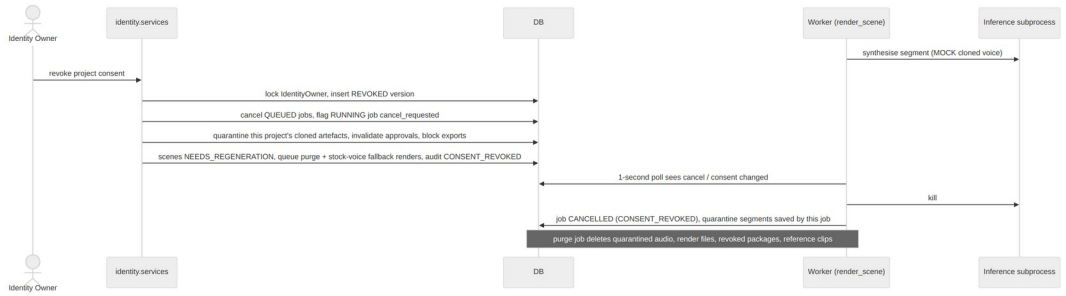


Figure 4.4: Sequence diagram: revocation while a cloned-voice job runs

4.5 State Diagrams

Long-running work and governance objects have explicit state machines, which are enforced in the service layer and covered by tests.

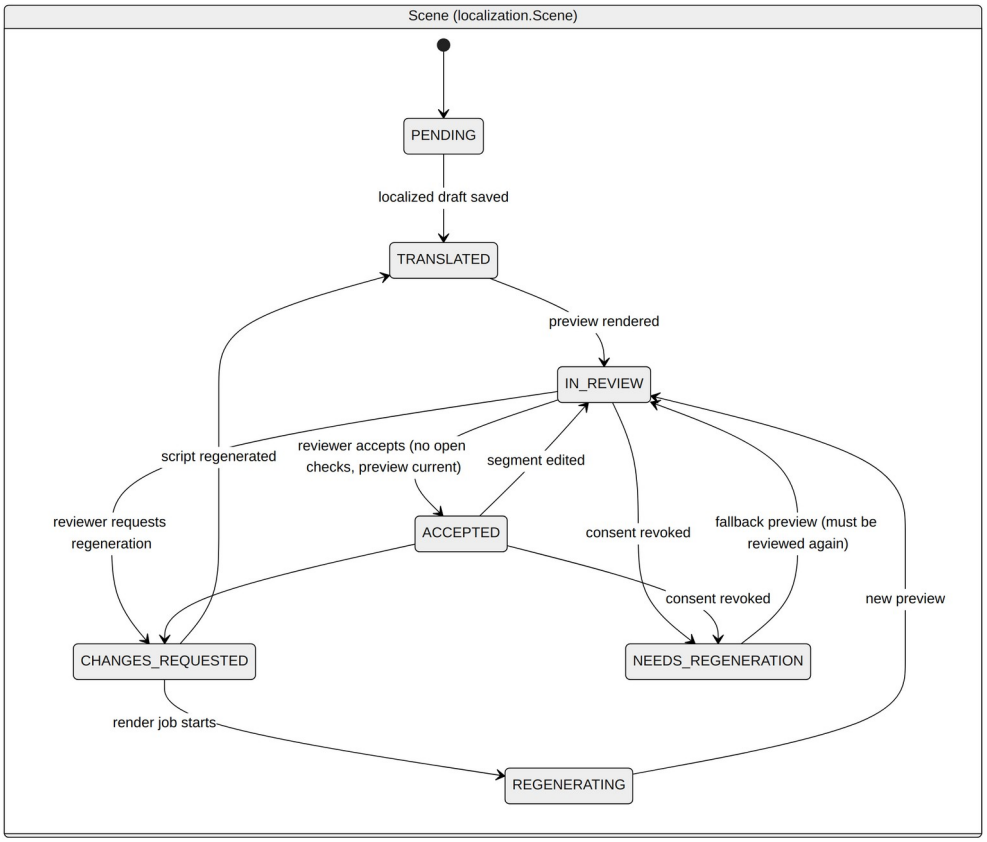


Figure 4.5: State diagram: Scene

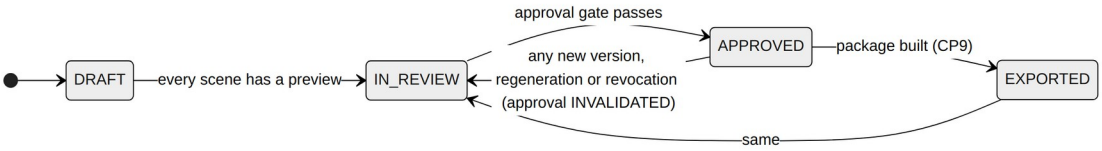


Figure 4.6: State diagram: Localization (project output)

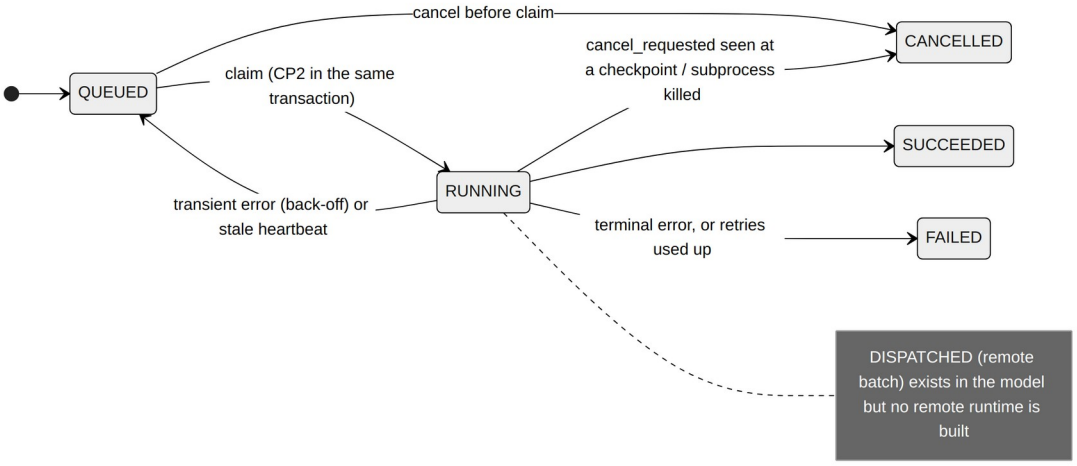


Figure 4.7: State diagram: Job

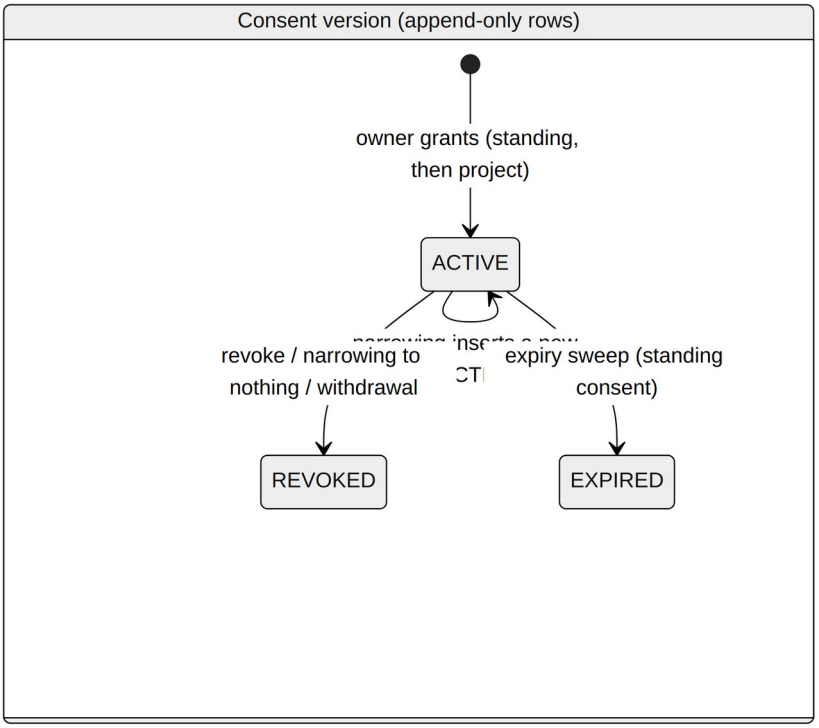


Figure 4.8: State diagram: Consent version (append-only rows)

4.6 Data Model

The data model (ADR 0014, amended by ADR 0020) has 28 model classes across the nine apps. Figure 4.9 shows the core entities and their relationships; the full class diagram, generated from the actual Django models by `scripts/generate_uml.py`, is in Appendix B. Key invariants are: translations, consent versions and audit records are **append-only**; an Approval stores an immutable **snapshot** of exact version ids; and every AI artefact records its adapter, model, prompt version and `is_mock` flag.

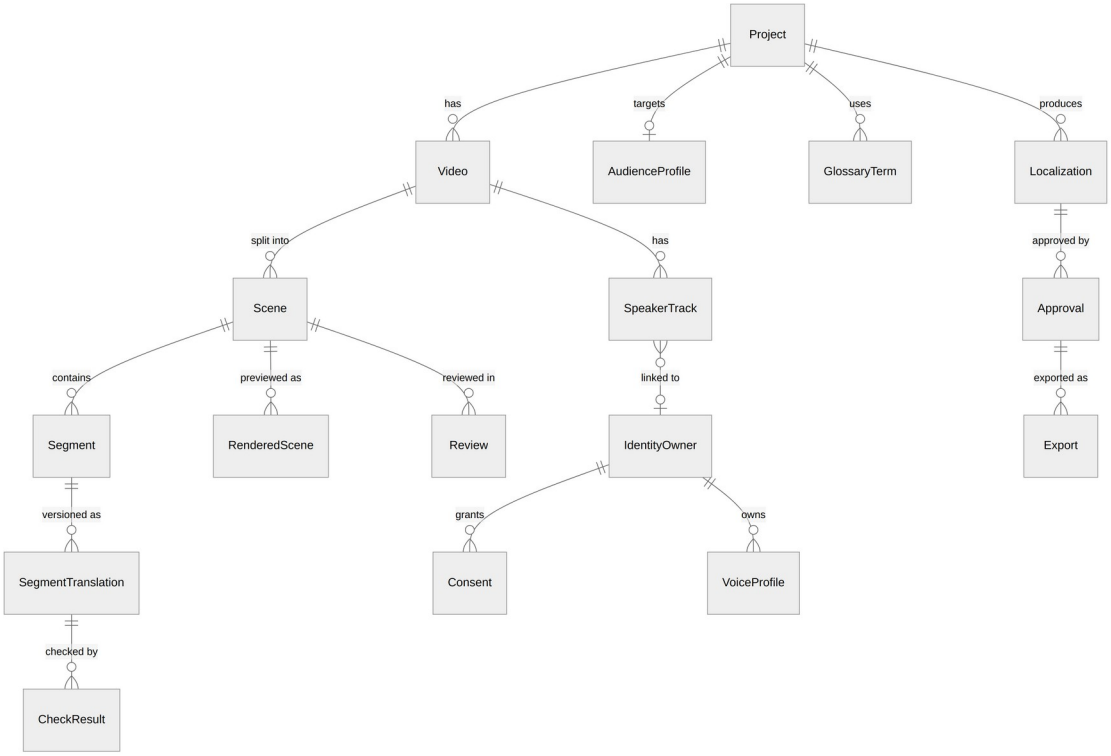


Figure 4.9: Core entity-relationship diagram

Table 4.2: Core entities

Entity	Purpose and key fields
Project	name, source_language, domain, status, data_classification, purpose, source_url / licence (published content)
Video	storage_key, sha256, duration_ms, codecs, size, transcript adapter/model, transcript_is_mock
Scene / Segment	index, start_ms, end_ms, status; segment source_text, asr_confidence, speaker_track
SegmentTranslation	version, subtitle_text, speech_text, adaptation_type, rationale, origin (AI/HUMAN), adapter, model, prompt_version, is_fallback, is_mock, context_inputs
CheckResult	kind (rule / AI flag), check_type, status (PASS / FAIL / UNVERIFIABLE …), resolution, reason, resolved_by
SynthesisedSegment / RenderedScene	audio_key, sha256, duration, rendering_mode, consent_version, tempo, intelligibility, is_mock
Review / RegenerationRequest / Approval	decision, issues, reviewed_versions; target, instruction; snapshot, status, invalidated_reason
IdentityOwner / Consent / VoiceProfile	withdrawn_at; kind (standing / project), version, uses, purposes, languages, processors, expires_at, status, evidence_ref; reference clip
Export	package_key, package_sha256, manifest, manifest_sha256, status
AuditRecord / AuditChainHead	seq, actor, action, entity, project, scene, consent_version, timestamp, details, prev_hash, hash
Job / ProviderConfig / ProviderUsage	type, lane, status, cancel_requested, attempts, heartbeat, payload; adapter chain per stage; usage ledger

4.7 Key Design Decisions

Each major decision was recorded as an ADR after comparing alternatives on quality, performance, reliability, privacy, cost, difficulty and ease of explanation. Table 4.3 summarises the most important ones; the full list is in Appendix A.

Table 4.3: Summary of key ADRs

Decision	Chosen option	Main reason
Architecture (0001)	Modular monolith, 9 apps	One developer, laptop demo; DB locks for consent
Database (0002)	PostgreSQL	Transactions, row locks, SKIP LOCKED, triggers
Job queue (0003)	Hand-written Postgres queue	Needs own cancel and consent checkpoints; no extra infrastructure
Providers (0005)	Interfaces + ordered chains + labelled mocks	Swappable, testable, ₹0
Consent (0006)	Backend gate at CP0–CP10, revocation cascade	Safety-critical: UI checks are not enough
Audit (0007)	Append-only SHA-256 hash chain	Tamper-evident, simple to verify
Checks (0008)	Rule-based hard checks + AI flags	Rules are explainable; AI only flags
Frontend (0010)	Server-rendered templates + htmx	No separate SPA build; simple
Published content (0021)	Gemini allowed only with recorded source URL and licence; MMS-TTS stock voice	Real output without sending personal data to a free tier
Quota safety (0022)	Batch per clip, no silent MOCK, 1.2× tempo cap, overrun check	Free-tier quota; honest failure

4.8 Consent Design

Consent has two levels: a **standing consent** recorded by the identity owner (uses, permitted purposes, languages, processors, expiry and a reference to the signed paper form), and a **project consent** that must stay within it. Consent rows are versioned and append-only. Only the identity owner can grant consent; an Administrator's grant is refused. The gate is re-checked at each stage boundary, as listed in Table 4.4.

Table 4.4: Consent checkpoints (ADR 0006)

Checkpoint	Where	On failure
CP0	Before source audio leaves the laptop (external ASR only)	Local ASR used; audited

Checkpoint	Where	On failure
CP1	API request that would use identity	Refused; audit CONSENT_DENIED
CP2	Job claim (same transaction)	Job CANCELLED
CP3	Before a reference clip is uploaded to a remote GPU	CANCELLED; nothing uploaded
CP4	Before synthesis of each segment	Stop; CANCELLED
CP5 / CP7	Saving synthesised audio / a rendered scene (row-locked)	Discarded
CP6	Before scene render / mix	CANCELLED
CP8	Approval: every consent version in the snapshot is still latest and ACTIVE	Approval refused
CP9	Before export build and when saving it (row-locked)	CANCELLED
CP10	Serving previews and downloads	HTTP 410 Gone with explanation

CP0 and CP3 are designed but have no code path yet, because no remote provider for audio or reference clips is integrated.

4.9 Audit Design

Every governance action is written by one function (`audit.services.record`). Each record stores the previous record's hash, and its own hash is `SHA-256(prev_hash + canonical_json(record))`. Appends are serialised by locking a single chain-head row. PostgreSQL triggers reject UPDATE, DELETE and TRUNCATE on the audit table. The `verify_audit_chain` command recomputes the whole chain, and every export manifest carries the current chain head. The design is **tamper-evident, not tamper-proof**: someone with full database rights could rebuild the entire chain, which is why the head is also copied into each exported package.

CHAPTER 5
IMPLEMENTATION

5.1 Technology Stack

The application is written in Python with Django 6.1.1 and PostgreSQL 17. Pages are server-rendered Django templates with htmx for partial updates (for example, the jobs panel refreshes every 2 seconds). ffmpeg and ffprobe handle validation, audio extraction, shot-cut detection, tempo adjustment, mixing and muxing. The code base contains about 10,100 lines of Python (excluding migrations), of which about 2,700 are in test files, plus about 570 lines of HTML templates.

5.2 Asynchronous Job Pipeline

Jobs are rows in a `Job` table with states QUEUED, RUNNING, SUCCEEDED, FAILED and CANCELLED (and DISPATCHED reserved for a future remote runtime). A worker claims the next job with `SELECT … FOR UPDATE SKIP LOCKED`, and the consent check CP2 runs in the same transaction. Transient errors are retried with back-off (5 s, 30 s, 120 s); a stale heartbeat returns the job to the queue. `cancel_requested` is a flag that workers check at every checkpoint. Separate lanes keep slow speech synthesis from blocking analysis and localization.

5.3 Video Analysis and Scene Planning

On upload, the file is validated with ffprobe **before** it is stored: container, codecs, a video stream, duration ≤ 5 minutes and size ≤ 500 MB. The analysis job extracts the audio and obtains the transcript from a human-made SRT/VTT file if supplied, otherwise from faster-whisper `small` on CPU, otherwise from the mock. Scene planning (`localization.services.plan_scenes`) then places a scene boundary between two segments when a shot cut (ffmpeg scene-change score) falls in the gap, when the pause is at least 1.5 s, or when the scene would exceed 30 s. Boundaries sit at the cut or the middle of the pause, and scenes cover the whole video. These thresholds are implementation choices and have not been tuned. Speaker diarization is not implemented: every segment is "Speaker 1".

5.4 Context-Aware Localization

For each segment the request carries: two neighbouring segments on each side, the speaker, the audience profile (locale, formality, reading level, cultural notes, declared unit conversions), the glossary, a **length budget** and any reviewer instruction. The budget (prompt version `loc-v3`, ADR 0022) is `max_speech_chars` = time available × the stock voice's measured speaking rate (11 characters per second for MMS-TTS Tamil, from a measured median of 11.95 on 20 lines); the model must keep every number, unit, negation, warning and instruction, and mark a line `TOO_LONG` if it cannot fit. The output has a subtitle text, a fully speakable speech text, an adaptation type and a stated rationale. Every saved draft records the context inputs used, the adapter, model and prompt version. Visual context is not used.

Quota-safe batching: because the Gemini free tier allows about 20 requests per day per model, a localize job sends the whole clip in one request when it fits (at most 80 segments and 12,000 characters), and the meaning check is one more request. If a real provider fails (quota, outage, bad output), the chain does **not** fall back to MOCK text: the scenes stay PENDING, the page shows "Translation pending … Nothing was machine-translated by a MOCK", and a Retry button is offered. Gemini is used only for projects classified as scripted team content or as published public content with a recorded source URL and licence (ADR 0021).

5.5 Critical-Content Checks

`localization/checks.py` implements a language-neutral framework: a source-language number reader and a target **language pack** (Tamil). The checks and their effects are summarised in Table 5.1.

Table 5.1: Critical-content checks (ADR 0008, ADR 0022)

Check	How it works	Result
Numbers	Digits, groupings, times and number words are read on both sides and compared as multisets. Tamil number words 0–100 are parsed (draft, to be verified by a Tamil rater)	FAIL on mismatch; UNVERIFIABLE where words cannot be read safely
Units	Lexicon with forms per language; a declared conversion makes the result UNVERIFIABLE	FAIL / UNVERIFIABLE / PASS
Glossary	Target forms matched by prefix, so inflected Tamil forms still match	A term with no target forms is UNVERIFIABLE, never PASS
Structure	Empty, untranslated, mostly Latin script for a non-	FAIL

Check	How it works	Result
	Latin target, over the reading-time budget	
Speech overrun	After real synthesis, a line that still needs more time than it has at 1.2× tempo (100 ms tolerance)	FAIL: "shorten the speech script"
AI meaning flag	MeaningCheckProvider flags possible meaning reversals (for example, a dropped negation)	Needs reviewer acknowledgement; never called verification

A FAIL or UNVERIFIABLE result blocks approval until the text is fixed or a reviewer overrides it with a written, audited reason. **Known limitation:** the numbers check compares counts, not positions, so swapping "2 tablets 3 times" into "3 tablets 2 times" still passes and must be caught by the reviewer.

5.6 Rendering and Timing Fit

`rendering.services.resolve_mode` decides the voice per speaker on the server: SUBTITLES_ONLY if policy forbids synthesis; otherwise CLONED_VOICE only with valid consent, an active reference clip, cloning allowed by policy and an available cloning adapter; otherwise STOCK_VOICE. The reason is shown to the reviewer. The stock voice is MMS-TTS Tamil on CPU; it refuses unspeakable text (digits or Latin letters) instead of silently dropping it. If speech is longer than its slot, the tempo is raised up to 1.2×. Each line is then transcribed back with faster-whisper and its character error rate (CER) is stored as a clearly labelled **intelligibility proxy** (not a listening test). The scene preview is an MP4 with the original picture, the original audio ducked under the new speech, and soft subtitles.

5.7 Consent Gate, Revocation and Mid-Job Stop

`identity.services.evaluate` is the only consent decider. Identity use is allowed only if the owner has not withdrawn; the latest project consent is ACTIVE, unexpired and includes the use and the language; it has a parent standing consent; the latest standing consent is ACTIVE, unexpired and permits the project's purpose; the project consent is within the standing consent; and the job is not flagged for cancellation. Saves lock the IdentityOwner row and re-run the gate before writing.

On revocation, one transaction sets affected scenes to NEEDS_REGENERATION, cancels render jobs, quarantines this owner's artefacts in this project, marks reference clips DELETED, invalidates only the approvals and exports that contain this owner's

identity, and queues a purge. After commit, stock-voice fallback renders are queued and must be reviewed again. A fixed lock order (IdentityOwner → Localization → Scene → Job → artefacts → Export → AuditChainHead) prevents deadlocks. While a synthesis subprocess runs, the worker polls the cancel flag and the consent gate about once a second and **kills** the subprocess on cancel; a test with a real subprocess passes within an 8-second bound (the actual stop time was not recorded). Exports that were already downloaded cannot be recalled, and the system says so.

5.8 Review, Approval and Export Gates

A reviewer's edit creates a new HUMAN version and re-runs the checks and the preview. Regeneration targets one scene only (its script, with an instruction such as "simpler words", or its preview). A scene can be accepted only with no open checks and a current preview. Approval (`review.services.approve`) is refused unless every scene is accepted, no blocking checks remain, every preview is current, consent CP8 passes, and the approver did not edit any segment in this version (**separation of duties**). Approval freezes an immutable snapshot; any later change invalidates it. Export is refused if the approval is invalid, if **any** artefact in the snapshot is MOCK, if any artefact is quarantined, or if CP9 fails. The package contains the localized MP4, SRT and VTT subtitles (tagged with the target language), `script.json`, `quality_report.json` and `evidence_manifest.json` (hashes, providers, rendering modes, consent versions, reviews, provenance, intelligibility proxy and the audit chain head).

5.9 Provider Abstraction and Mocks

Each AI stage (transcription, localization, meaning check, stock voice, cloned voice) is an interface with an ordered chain of adapters that the Administrator can reorder. Mocks exist only in development and test modes, are named as mocks, carry `is_mock`, show a MOCK badge in the UI, and can never be exported (Table 5.2). A free-quota ledger warns the Administrator at 80% and 100% of the limits they enter.

Table 5.2: Provider chains and labelled mocks

Stage	Real / candidate adapter	Mock (dev/test only)	Status
Transcription	Human SRT/VTT; faster-whisper small (CPU)	MockTranscriptionProvider: "[MOCK] Transcript segment N."	Candidate run on real clips
Localization	Gemini (gemini-3.5-flash,	MockTranslationProvider: copies	Candidate;

Stage	Real / candidate adapter	Mock (dev/test only)	Status
	prompt loc-v3)	source, applies glossary, prefix "[MOCK]"	quality not measured
Meaning check	Gemini batch check	MockMeaningCheckProvider: negation keyword rule (not AI)	Candidate
Stock voice	MMS-TTS Tamil (CPU)	MockStockVoiceProvider: 440 Hz tone	Candidate, accepted in ADR 0021
Cloned voice	None yet (ADR 0018 pending)	MockClonedVoiceProvider: 660 Hz tone, imitates no one	MOCK only

5.10 User Interface

Each role sees its own pages. The screenshots below were taken in headless Chromium from a throwaway seeded database on 2026-10-12 and show MOCK output (the purple banner and MOCK badges are part of the honest-labelling design).

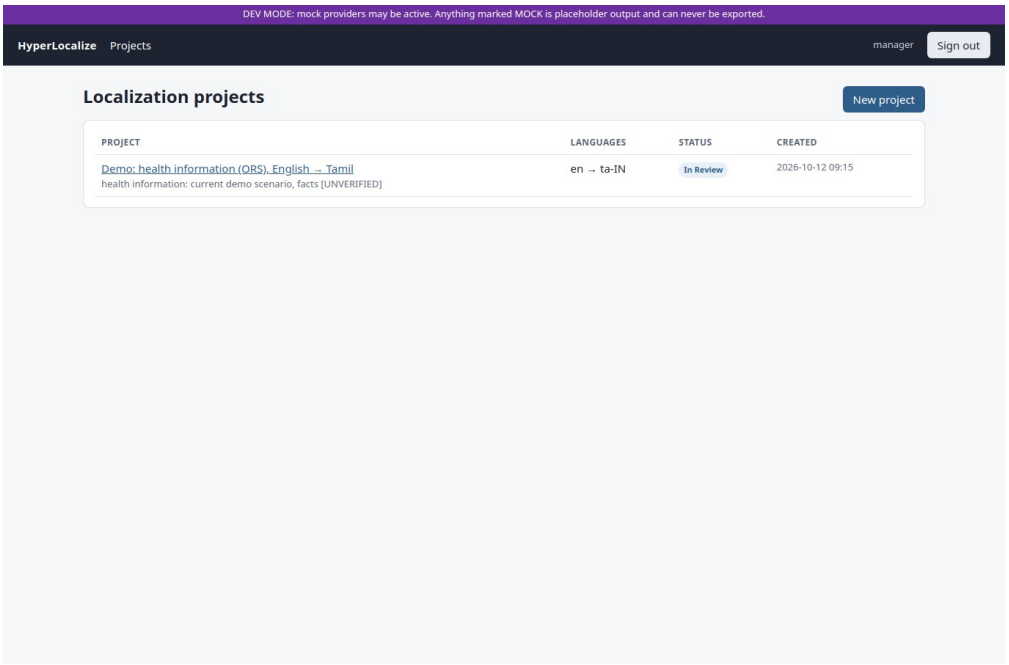


Figure 5.1: Localization Manager: project list

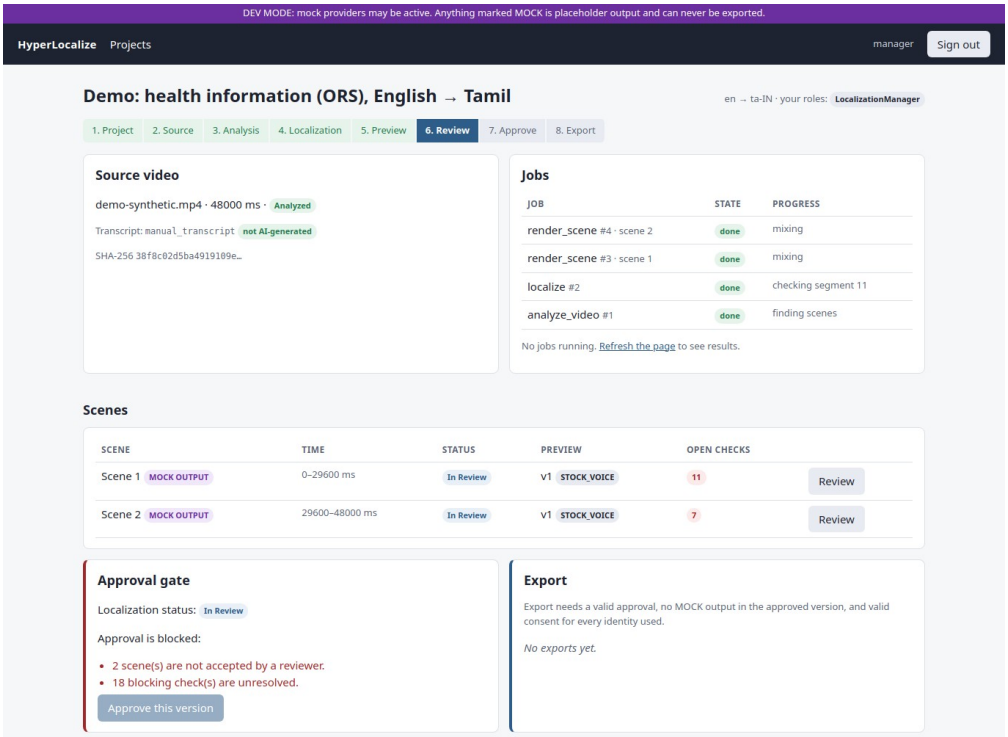


Figure 5.2: Localization Manager: project detail with approval gate

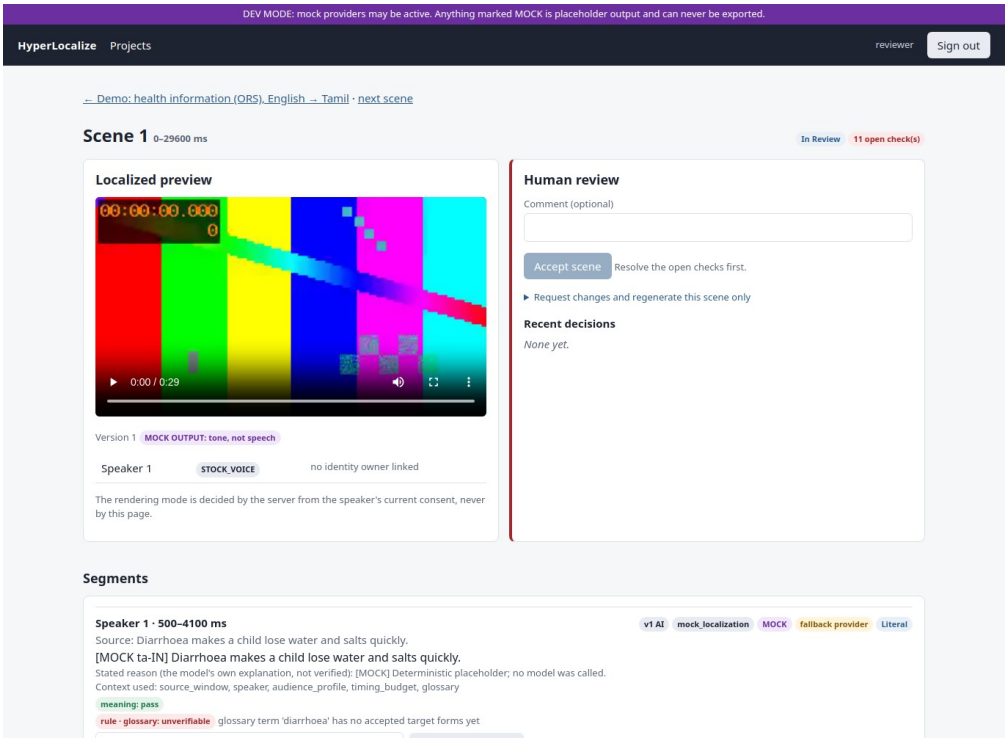


Figure 5.3: Reviewer: scene review with preview, checks and override fields

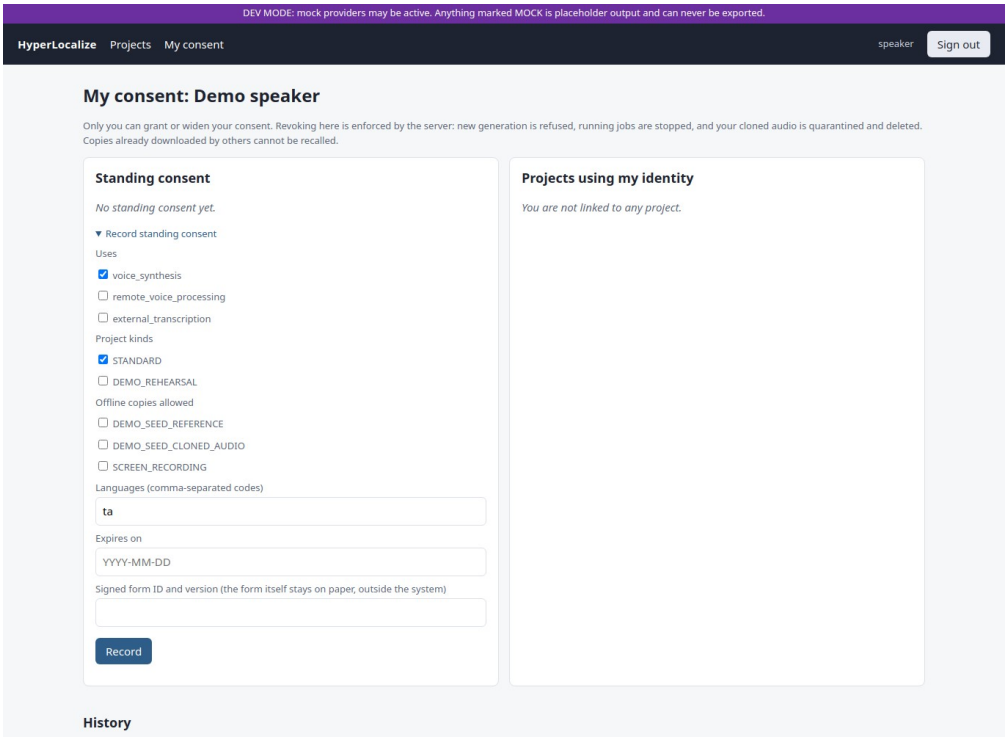


Figure 5.4: Speaker / Identity Owner: consent page

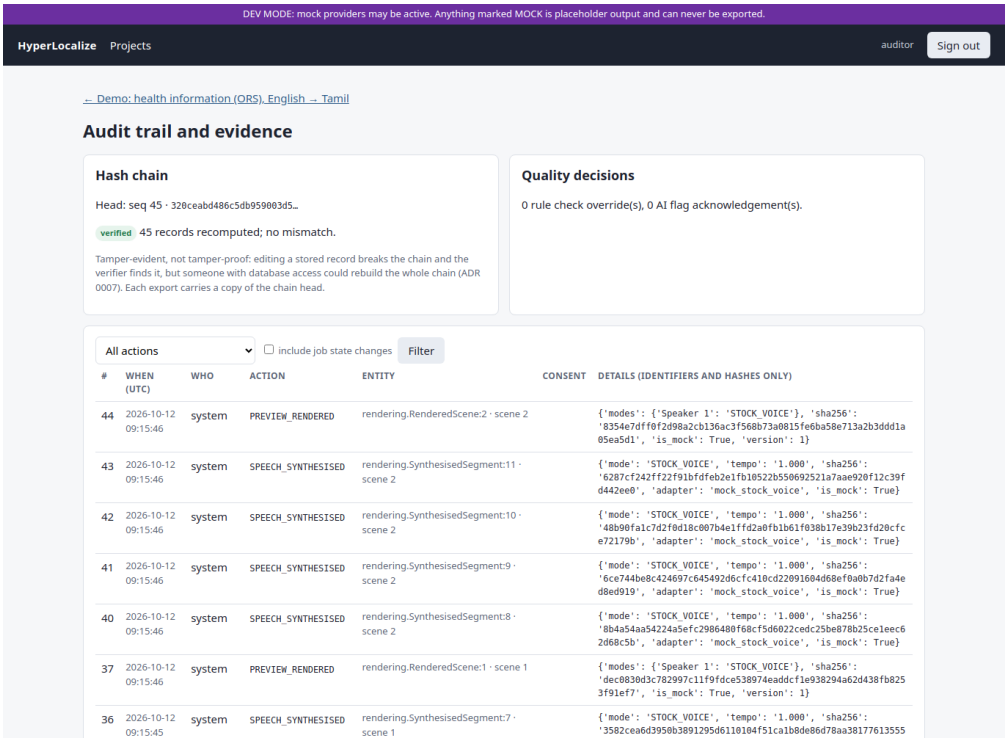


Figure 5.5: Auditor: audit trail and chain verification

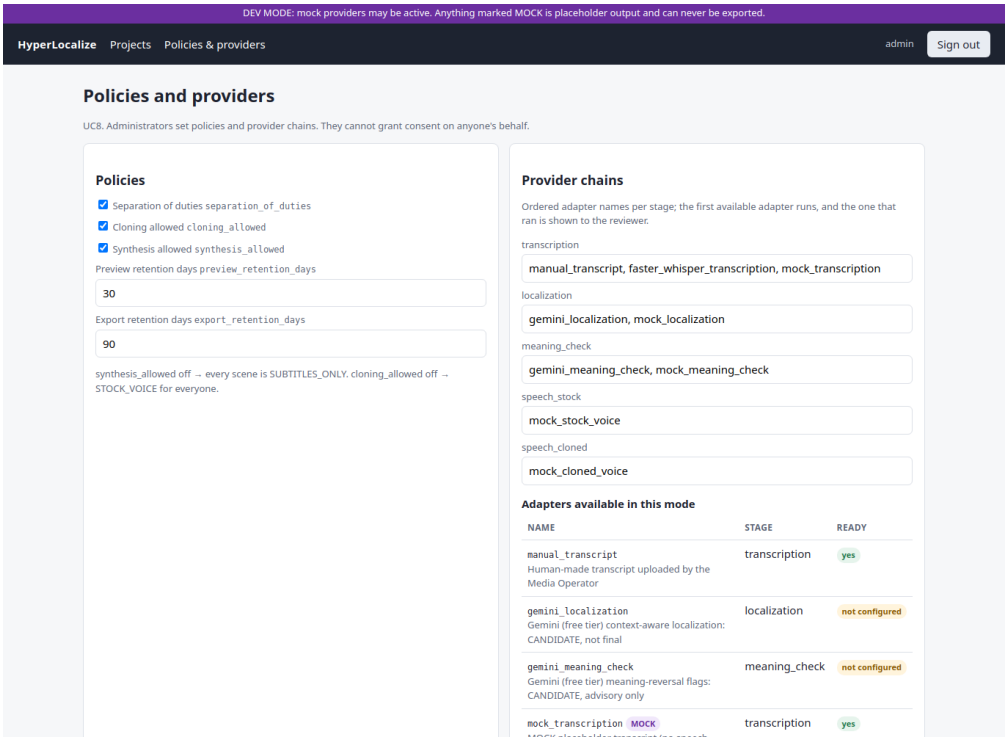


Figure 5.6: Administrator: policies, provider chains and quota warnings

CHAPTER 6
TESTING

6.1 Testing Strategy

Testing follows the project rule that every important component is tested and that consent enforcement and state transitions have explicit tests, written **negative-first** (every way the gate must refuse is tested before the way it allows). The suite runs on real PostgreSQL 17 and real ffmpeg, with MOCK AI providers and synthetic media; since ADR 0022 the test runner forces test mode so that no test can reach the real Gemini API.

- **Unit tests:** checks, number parsing, scene planning, provider chains, quota logic, audit chain.
- **Integration tests:** full workflows through the service layer (upload → analysis → localization → preview → review → approval → export), identity workflows and revocation.
- **Web-layer tests:** pages per role, 403 for outsiders, wrong-role POSTs, previews and CP10 410 after revocation, login rate limit.
- **End-to-end test:** `scripts/e2e_check.py` drives a live server and worker over HTTP.
- **Black-box tests:** equivalence partitioning and boundary value analysis on the check functions.

6.2 Test Areas

Table 6.1: Test areas

Area	File	What is proven
Audit chain	audit/tests.py	Chain links and verifies; UPDATE, DELETE, TRUNCATE rejected by the database; tampering with the trigger disabled is detected; no raw content stored
Consent gate	identity/tests.py	No consent, no standing consent, Administrator grants, wider-than-standing, wrong use or language, revoked, expired, old version after revocation, narrowing, final withdrawal, expiry sweep
Workflow	tests/ test_workflow.py	Scenes at shot cuts; context recorded; checks block acceptance; overrides need reason and role; scene-only regeneration; approval gate; separation of duties; MOCK never exportable; edit after approval invalidates it
Identity in workflow	tests/ test_identity_workflow.py	Cloned mode with consent; stock fallback without; revocation cascade quarantines, purges, falls back; scope isolation; real mid-job kill of the inference subprocess

Area	File	What is proven
Review fixes	tests/ test_review_fixes.py	Regressions for the independent code review (1 blocker, 6 majors): two-owner mid-job stop, withdraw vs reset in both orders, and more
Demo reset	tests/ test_demo_reset.py	Administrator only; rebuild only appends; refusals for purpose, drift, withdrawal
Web layer	tests/test_views.py	Per-role pages, 403s, CP10 410 after revocation, admin-only policies, login rate limit, retention sweeps
Providers and quota	pipeline/tests.py, tests/test_quota.py	Mocks absent in demo mode; chain fallback; data-use rule; Gemini parsing against a fake HTTP layer; 80% / 100% quota boundaries
Checks	localization/tests.py	Numbers (digits, words, Tamil digits, Indian grouping), units, conversions, glossary, structure, scene planning
ADR 0021/0022, governance	tests/ test_published_public.py, test_adr0022.py, test_governance.py	Provenance rules; batching; no silent MOCK; overrun check; Tamil number words; agent-review withdrawal

6.3 Sample Test Cases

Table 6.2: Sample test cases (all part of the passing suite)

ID	Test case	Input / condition	Expected result	Result
TC01	Upload validation	Corrupt or unsupported file	Rejected before storage	Pass
TC02	Number mismatch	Source "Give 2 sachets", target with 5	Numbers check FAIL, approval blocked	Pass
TC03	Override needs reason	Override without a reason, or by a non-reviewer	Refused	Pass
TC04	Scene acceptance	Accept with open checks	Refused	Pass
TC05	Separation of duties	Manager who edited a segment approves	Refused; another manager must approve	Pass
TC06	MOCK export	Approved snapshot contains MOCK output	Export refused	Pass
TC07	Administrator grants consent	Admin tries to grant on behalf of a speaker	Refused	Pass
TC08	Revocation mid-job	Revoke while a cloned-voice subprocess runs	Subprocess killed, job CANCELLED within the 8 s bound	Pass
TC09	Preview after revocation	Request a revoked preview	HTTP 410 Gone	Pass
TC10	Audit tampering	UPDATE/DELETE an audit row; edit with trigger disabled	Rejected; verifier detects change	Pass
TC11	Provider quota failure	Gemini returns 429 daily quota	Scenes PENDING, no MOCK text, Retry shown	Pass

ID	Test case	Input / condition	Expected result	Result
TC12	Edit after approval	Reviewer edits a segment after approval	Approval INVALIDATED	Pass

6.4 Black-Box Testing

A separate black-box suite (`blackbox_testing/test_hyperlocalize_blackbox.py`) tests the check functions only through their inputs and outputs:

- **Equivalence partitioning:** a correct translation with matching numbers (valid class); a changed number, a missing number, an empty target and an untranslated copy of the source (invalid classes); and a language pair with no verification pack (unsupported class → UNVERIFIABLE).
- **Boundary value analysis:** the reading-time budget for a 1-second line (`budget = duration_s × 18 + 10` = 28 characters: 28 passes, 29 fails); the Latin-script share limit (exactly 50% passes, 60% fails); and a single-character translation.

6.5 Test Results

Table 6.3: Measured test results

Suite	Date	MEASURED result
Full application suite (`manage.py test hyperlocalize`)	2026-10-22	**210 passed, 2 skipped** (optional real-model tests), 125 s
Black-box suite (pytest)	2026-10-22	**11 passed**
End-to-end over HTTP (`scripts/e2e_check.py`)	2026-10-06	**15 / 15 steps passed**: consent, cloned (MOCK) preview, review, approval, MOCK export refused, revocation (410, approval invalidated, fallback, clip deleted), subtitles-only export, auditor verifies chain
Spike evaluation tooling (`spike/`)	2026-10-05	75 passed, 1 skipped (synthetic inputs only)
UML render check	2026-10-12	All 10 Mermaid diagrams render

The test count grew from 102 to 210 over the build as features and review fixes were added. An independent code review of the consent and audit code found 1 blocker and 6 major issues; all were fixed with regression tests.

6.6 Not Yet Tested

- Any real AI model on real data at evaluation scale (only the single-clip trial in Chapter 7).
- The Docker images (Docker is not installed on the development laptop).
- Browser interaction beyond page loads (forms were not submitted in a browser test).
- Performance and load.

6.7 Governance of Test Runs

A project rule states that the AI assistant used in development may create test projects and run analysis, localization and previews, but **never** accepts scenes, overrides checks, approves or exports through any user account. On 2026-10-22 an earlier test export (clip 01, project 4) had been reviewed and approved by the assistant through the demo accounts; it was withdrawn and audited as `AGENT_REVIEW_WITHDRAWN`, and is never cited as human-reviewed. This rule is a process rule; the application cannot distinguish an assistant from a person using the same account.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Evaluation Plan

Quality metrics and their thresholds were fixed in `docs/SPIKE_PLAN.md` **before** any results existed, to avoid tuning the criteria to the results. Table 7.1 lists every metric with its TARGET and MEASURED RESULT.

Table 7.1: Evaluation metrics and MEASURED results (targets are fixed in SPIKE_PLAN §5)

Metric	Definition	MEASURED RESULT
M1 Transcription accuracy	WER / critical-token errors on the test clip	NOT MEASURED (no consented recording yet)
M2 Translation quality	MQM points per 100 words; pairwise preference; chrF	NOT MEASURED (needs a Tamil-literate rater)
M3 Rule-check catch rate	Share of seeded errors caught	NOT MEASURED
M4 Speech quality and similarity	Naturalness; pronunciation; speaker similarity	NOT MEASURED. Proxy only (clip 01, stock voice): back-transcription CER 0.16 over 850 characters
M5 Timing fit	Share of segments within ±15% of their slot	Pre-spike, clip 01: 2 / 14 lines
M6 Speed	Model load, per-segment time, peak RAM	NOT MEASURED
Consent enforcement	Identity use refused in every denied state; revocation stops a running job	All tested denied states refused; mid-job stop within the 8 s test bound (automated tests)
Review gate integrity	No approval with open checks, unaccepted scenes, stale previews or invalid consent; no MOCK export	0 bypasses in the tested cases
Audit integrity	Edits rejected; tampering detected	Rejected and detected in the tested cases
Context ablation	Context on vs off vs plain MT	NOT MEASURED

7.2 First Real-Clip Trial (n = 1)

On 2026-10-22 one public video was run end to end: `test_datasets/01_handwashing_sf_health_dept_en` (58.4 s, CC BY 3.0, San Francisco Government Television), classified as published public content with its source URL, licence and attribution recorded. The pipeline was faster-whisper `small` (CPU) → Gemini `gemini-3.5-flash` (prompt `loc-v2`) → MMS-TTS Tamil (CPU)

→ review → approval → export. This is **directional evidence on one clip, not a quality evaluation**, and the review in this run was done by the AI assistant, not a human linguist (that export was later withdrawn; Section 6.7).

- 14 of 14 lines were translated by Gemini, after two fallbacks on scene 2 (HTTP 429 per-minute limit, then HTTP 503).
- The numbers 1, 2, 3, 20 and 60% and the negation ("don't have soap") were kept; minor issues were noted ("rinse" translated as "wash", one transliterated term, a US web address kept for a Tamil audience). This reading is [UNVERIFIED] until a Tamil-literate rater checks it.
- **Timing fit:** only **2 of 14 lines** fitted their slot; total Tamil speech was 79.1 s against 58.0 s of slots, with a tempo cap of 1.15× at the time (Figure 7.1).
- **Intelligibility proxy:** Whisper back-transcription CER **0.16 over 850 characters** (corrected from an earlier 0.17 over 503, whose normalisation dropped Tamil vowel signs). The proxy is pessimistic because it includes Whisper's own errors.
- **Speaking rate** of MMS-TTS Tamil: median 11.95 characters per second over 20 lines (range 9.4–14.6), used to calibrate the length budget.

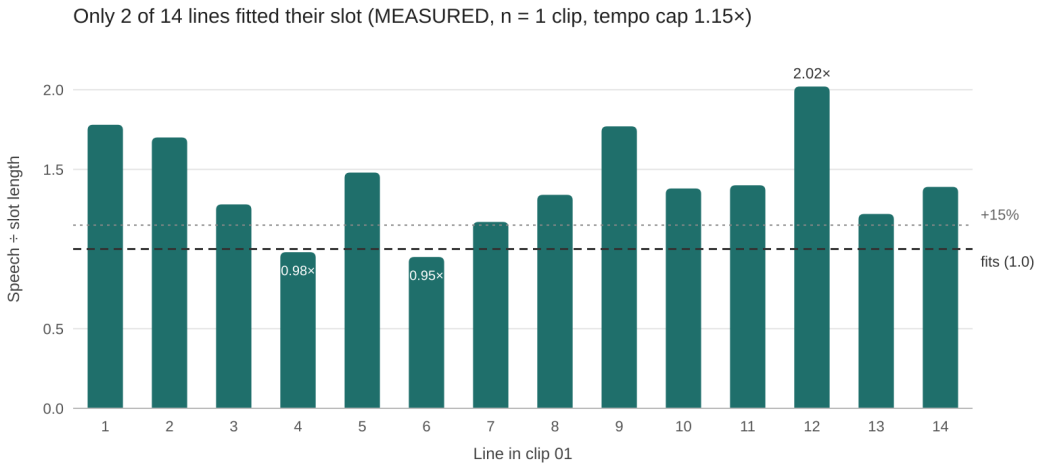


Figure 7.1: Clip 01 timing: Tamil speech length ÷ slot length per line (1.0 = fits)

7.3 Design Changes Driven by the Results

The trial exposed four problems, all addressed in ADR 0022 and covered by tests:

Table 7.2: Problems found and design changes (ADR 0022)

Problem observed	Change made
One meaning-check call per line exhausted the free tier; on 429/503 the chain silently fell back to MOCK text	Batch the whole clip into one localization request and one meaning-check request (about 16 → 2 requests for clip 01); never fall back to MOCK after a real provider fails; show "translation pending" with Retry

Problem observed	**Change made**
Only 2 / 14 lines fitted their slot	Length budget in the prompt (11 chars/s × time available); tempo cap 1.2×; blocking speech-overrun check
Export tracks tagged `und`	Audio and subtitle tracks tagged with the target language (`tam`)
Tamil number words were unverifiable	Draft parser for Tamil number words 0–100; a wrong number word now FAILS

A re-run of the same clip with these changes (project 5) stopped honestly at "translation pending" because the Gemini daily quota had already been used: 0 translations and 0 MOCK fallbacks. It is waiting for the quota to reset and then for **human** review. The improvement in timing fit from ADR 0022 has therefore **not yet been measured**.

7.4 Discussion

The results support the workflow's governance claims more strongly than its quality claims. The consent, review-gate and audit properties are backed by automated tests that try to break them. The quality of real AI output, by contrast, has only one-clip directional evidence, and the first real run immediately revealed timing and quota problems that a pure pipeline would have shipped silently. That is itself the main argument for the design: the system made the problems visible, blocked them at the gate, and recorded them.

The free-tier constraint shaped the engineering significantly: about 20 requests per day forced batching and an explicit pending state rather than silent degradation. The tests also uncovered that earlier test runs had been sending real requests with the developer's key; test mode is now forced by the runner.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Conclusion

HyperLocalize demonstrates that video localization can be built as a **controlled, accountable workflow** around commodity AI models. All nine use cases are implemented and tested on the MVP path: context-aware drafting with recorded context, rule checks on critical content that block approval, scene-level review and regeneration, a backend consent gate with a revocation cascade that kills running jobs, an approval gate with separation of duties and an immutable snapshot, an export that refuses placeholder output, and a hash-chained audit trail carried into an evidence manifest. 210 automated tests pass. Real candidate providers produce real Tamil speech and subtitles from a real public video.

Equally important is what is **not** yet shown: translation quality, speaker similarity, speed and the benefit of context have not been measured, voice cloning is still a mock, and the timing fix has not been re-measured. The project reports these gaps explicitly rather than hiding them.

8.2 Limitations

- No formal evaluation yet: M1–M6 and the context ablation are NOT MEASURED; the only real-data result is one 58-second clip.
- Cloned voice is MOCK (ADR 0018 pending); identity beyond voice (face, likeness) is out of scope.
- No speaker diarization (every segment is "Speaker 1"); no visual context.
- The numbers check compares counts, not positions; Tamil number-word, unit and glossary forms are drafts until a Tamil-literate rater reviews them.
- A meaning-check network call can run inside the consent lock during a demo reset (open review finding).
- The quota ledger counts only successful calls; the Gemini free tier allows about 20 requests per day per model.
- Docker Compose files are written but have not been run; the demo runs with a development script.
- The governance rule against assistant review is a process rule, not enforced in code. MMS-TTS is licensed for non-commercial use only.

8.3 Future Work

1. Run the planned evaluation (spike) with consented Speakers A and B and Tamil-literate raters R1 and R2: WER, MQM, chrF, seeded-error catch rate, listening tests, timing fit and speed; record results in `docs/EVALUATION.md`.
2. Measure whether context helps, with an ablation: context on vs context off vs a plain machine-translation baseline (IndicTrans2, once access is granted).
3. Choose the cloned-voice provider (ADR 0018) from measured results, or keep stock voice and subtitles only if cloning fails the thresholds.
4. Re-measure timing fit after ADR 0022 and add a Whisper baseline on natural Tamil speech for the CER proxy.
5. Fix open findings: move the meaning check out of the consent lock, count every HTTP attempt in the quota ledger, and compare number positions as well as counts.
6. Add speaker diarization and visual context (keyframes, on-screen text OCR); more language pairs and several target languages per project.
7. Run and document the Docker Compose deployment; extend the browser test to submit forms.
8. Lip-sync and face processing remain future scope and would need their own consent design.

8.4 Viva Summary

"HyperLocalize is a general, controlled video localization workflow. A video is analysed into scenes and segments, and each segment is drafted using its context. The draft is never trusted: numbers, units and key terms are rule-checked, and a failure blocks approval. A reviewer edits, overrides with a reason, or regenerates one scene. A manager approves a frozen snapshot. A speaker's voice is used only while their own consent is valid, and revoking it stops a running job within seconds. Every action goes into a hash-chained audit log, and the export carries the chain head as evidence: tamper-evident, not tamper-proof."

REFERENCES

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv:2212.04356, 2022.

[2] J. P. Gala et al., "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages," Transactions on Machine Learning Research, 2023.

[3] V. Pratap et al., "Scaling Speech Technology to 1,000+ Languages," arXiv:2305.13516, 2023.

[4] J. Kim, J. Kong and J. Son, "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech," in Proc. ICML, 2021.

[5] A. Lommel, H. Uszkoreit and A. Burchardt, "Multidimensional Quality Metrics (MQM): A Framework for Declaring and Describing Translation Quality Metrics," Tradumàtica, no. 12, 2014.

[6] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in Proc. Tenth Workshop on Statistical Machine Translation, 2015.

[7] Coalition for Content Provenance and Authenticity (C2PA), "C2PA Technical Specification," c2pa.org.

[8] M. Nygard, "Documenting Architecture Decisions," 2011 (the ADR format used in this project).

[9] Django Software Foundation, "Django documentation," docs.djangoproject.com.

[10] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation: SELECT … FOR UPDATE SKIP LOCKED; CREATE TRIGGER," postgresql.org/docs/17.

[11] SYSTRAN, "faster-whisper," github.com/SYSTRAN/faster-whisper.

[12] Meta AI, "facebook/mms-tts-tam" model card (CC-BY-NC-4.0), huggingface.co.

[13] Google, "Gemini API documentation and free-tier terms," ai.google.dev.

[14] htmx, "htmx documentation," htmx.org.

[15] FFmpeg project, "FFmpeg documentation (ffprobe, scene detection, atempo)," ffmpeg.org.

[16] Wikimedia Commons: source pages and licences of the 12 test videos, listed in `test_datasets/README.md`.

APPENDIX A: ARCHITECTURE DECISION RECORDS

ADR	Title	Status
0001	Modular monolith (9 Django apps) with one worker process	Accepted
0002	PostgreSQL as the primary database	Accepted
0003	Hand-written Postgres job queue with explicit job states	Accepted
0004	Media storage behind an interface, local filesystem for the MVP	Accepted
0005	AI provider abstraction: free provider chains, mock providers, zero-spend policy	Accepted
0006	Consent gate, checks at each stage boundary, revocation cascade	Accepted
0007	Append-only, hash-chained audit records	Accepted
0008	Critical-information verification: rule-based hard checks + AI flags	Accepted
0009	Backend framework: Django	Accepted
0010	Frontend: server-rendered templates + htmx	Accepted
0011	Authentication and authorisation: sessions + project-scoped roles	Accepted
0012	Deployment: Docker Compose on a laptop, demo-resilience plan	Accepted
0013	Use-case boundaries and actor responsibilities	Accepted
0014	Core data model	Accepted
0015	Identity definition: consent-governed voice identity with fallback	Pending evaluation
0016	Context-aware translation method and provider	Pending evaluation
0017	Transcription (English ASR) provider	Pending evaluation
0018	Tamil speech synthesis: stock voice and cloned voice	Pending evaluation
0019	Kaggle Notebooks as a remote GPU batch runtime	Pending evaluation
0020	Demo reset by rebuild, offline-copy register, identity withdrawal	Accepted
0021	Published public content (Gemini allowed, provenance required) and MMS-TTS Tamil stock voice	Accepted, implemented
0022	Quota-safe batching, timing fit, Tamil track tags, number words, intelligibility proxy	Accepted, implemented

APPENDIX B: CLASS DIAGRAM

Generated from the actual Django models by `scripts/generate_uml.py` (source: `docs/uml/class_diagram.md`).

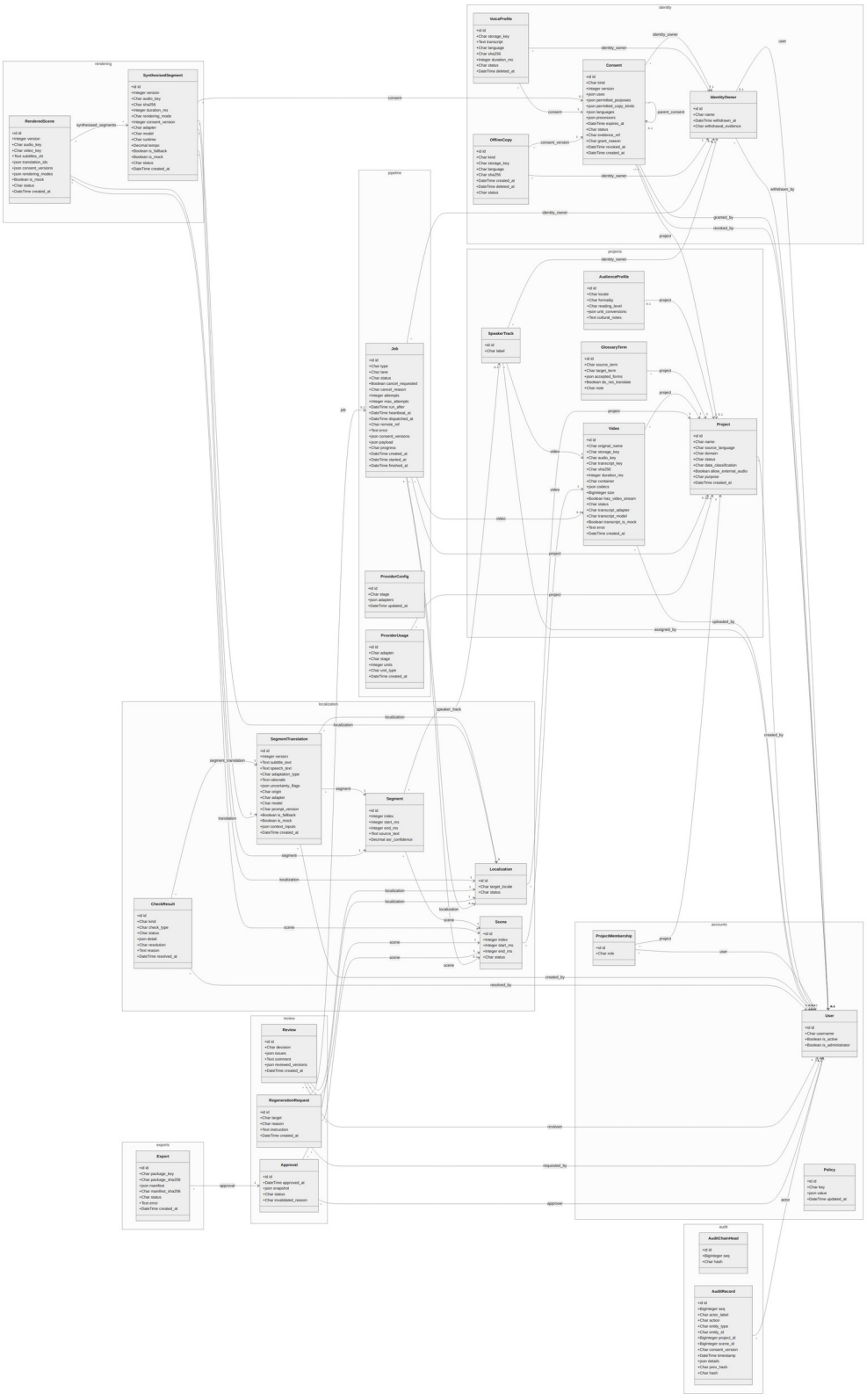


Figure B.1: Full class diagram (generated from the code)

APPENDIX C: USE-CASE STATUS

Use case	Status (2026-10-22)	Notes
UC1 Create and Generate a Localization Project	Implemented (tested)	Create, configure audience and glossary, add members, start the pipeline
UC2 Upload and Analyze Source Video	Implemented (tested)	ffprobe validation, limits, audio extraction, shot cuts, scenes; faster-whisper candidate ran on real clips; no diarization
UC3 Generate Context-Aware Localization Preview	Implemented with MOCK; real Gemini candidate partial	Context inputs recorded; visual context not implemented; quality not evaluated
UC4 Generate Localized Preview	Implemented with stock voice; cloned voice MOCK	MMS-TTS Tamil; timing fit poor before ADR 0022
UC5 Review and Regenerate a Scene	Implemented (tested)	Human versions, overrides, acknowledgements, scene-only regeneration
UC6 Approve and Export Localization Package	Implemented (tested)	Gate, snapshot, separation of duties, MOCK refused, evidence manifest
UC7 Revoke Consent and Stop Identity	Implemented with MOCK voices	Cascade, mid-job kill tested, purge, withdrawal, expiry sweep
UC8 Administer Users, Providers and Policies	Implemented (tested)	Policies and provider chains, quota warnings; users via Django admin
UC9 Inspect Quality, Audit and Evidence	Implemented (tested)	Audit filters, chain verification, overrides list

APPENDIX D: RUNNING THE SYSTEM

Development setup on Linux, without Docker:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements-
dev.txt
cp .env.example .env
scripts/run_dev.sh      # dev PostgreSQL 17, migrations, a worker,
web server on 127.0.0.1:8000
.venv/bin/python manage.py seed_demo --password '<choose-a-password>'
.venv/bin/python manage.py test hyperlocalize      # run the test
suite
```

Demo accounts: `manager`, `manager2` (second manager for separation of duties), `reviewer`, `operator`, `auditor`, `speaker`, `admin`. Optional real candidates: `pip install -r requirements-ml.txt` (faster-whisper; MMS-TTS after installing CPU PyTorch and `manage.py download_models --mms-tts`), and `HL_GEMINI_API_KEY` in `.env`. Useful commands: `check_providers`, `build_demo_seed`, `demo_reset`, `verify_audit_chain`.

APPENDIX E: ABBREVIATIONS

Term	Meaning
ADR	Architecture Decision Record
ASR	Automatic Speech Recognition
CER / WER	Character / Word Error Rate
CP0–CP10	Consent checkpoints (ADR 0006)
LLM	Large Language Model
MMS	Massively Multilingual Speech (Meta)
MQM	Multidimensional Quality Metrics
MT	Machine Translation
MVP	Minimum Viable Product
MOCK	A clearly labelled placeholder provider used only in development and tests
ORS	Oral Rehydration Salts (demo topic)
SRT / VTT	Subtitle file formats
TTS	Text-to-Speech
UC	Use Case
UML	Unified Modeling Language